Optimistic vs Pessimistic Locking

version checks, FOR UPDATE, and atomic updates

Watch the 3-min video

In one line: read-modify-write in application code loses updates under concurrency. Pessimistic locking locks the row first (FOR UPDATE); optimistic locking checks a version on write and retries; for simple counters, one atomic conditional UPDATE beats both.

1 The problem: the flash sale

A flash sale: **100 pairs** of a sneaker and **10,000 shoppers** clicking buy in the same second. Read the stock, subtract one, write it back, and you will sell far more than 100. That is the **lost update** problem.

The bug in miniature: two requests both read a stock of 1. Both subtract one in application code, and both write 0. Two orders, one sneaker.

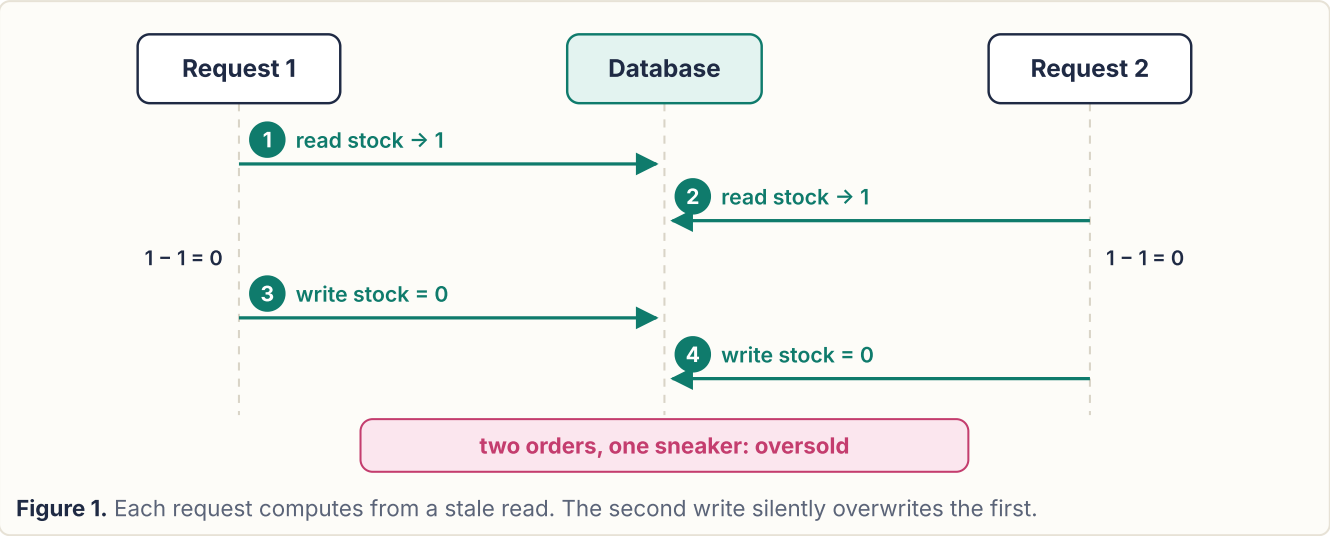


Figure 1. Each request computes from a stale read. The second write silently overwrites the first.

2 Pessimistic locking: lock first

Pessimistic locking **assumes conflicts will happen**, so it locks before reading. `SELECT … FOR UPDATE` takes an exclusive lock on the product row. The second request waits until the first commits, then reads the **updated** stock.

It is correct and simple. But under heavy contention, requests **queue up behind the lock**, and long transactions hold it even longer.

3 Optimistic locking: check on write

Optimistic locking **assumes conflicts are rare**, so it takes no lock at all. Each row has a `version` column. Read the row and its version, then update **only if the version is still the one you read**:

```
-- read: stock = 1, version = 7
UPDATE products
   SET stock = 0, version = version + 1
 WHERE id = 9 AND version = 7;
-- 1 row updated  → success
-- 0 rows updated → someone changed it first: re-read and retry
```

This is **compare-and-set**: the database checks and writes in one atomic step.

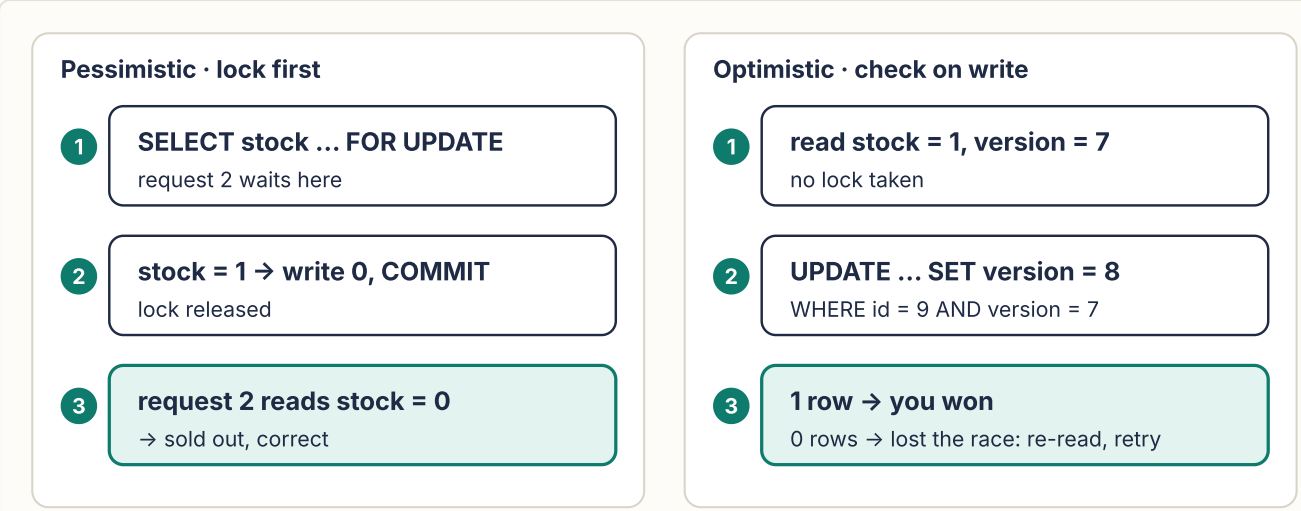


Figure 2. Pessimistic makes the second request wait; optimistic lets both run and makes the loser retry.

4 Simplest of all: one atomic, conditional update

For a simple counter like stock, skip the separate read entirely:

```
UPDATE products SET stock = stock - 1
 WHERE id = 9 AND stock > 0;
-- 1 row  → you got a pair
-- 0 rows → sold out
```

The database applies it atomically, holding the row lock only for that one statement (in autocommit). No separate read, no lock held across your code, and **no overselling**.

WORKED EXAMPLE: 100 PAIRS, 10,000 CLICKS

- **Read-modify-write:** many requests read the same stock value, so the shop can sell well over 100 pairs.
- **Optimistic:** correct, but for each sale, many of the requests that read the same version lose and retry, again and again: a retry storm on one row.
- **FOR UPDATE:** correct; requests wait in line. If each holds the lock for 5 ms, the row handles at most ~200 purchases per second.
- **Atomic conditional update:** correct, and the lock lasts only for one statement. Exactly 100 calls see “1 row”; the other 9,900 see “0 rows” and get “sold out”.

5 How to choose, and the HTTP version

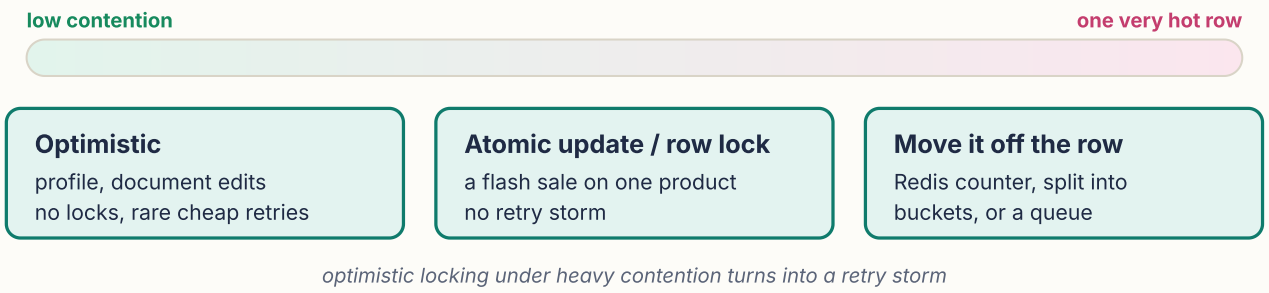


Figure 3. Pick by contention. For extremely hot items, move the counter to Redis, split inventory into buckets, or funnel purchases through a queue.

Optimistic locking works across HTTP too. The server returns an `ETag` (a version) with the resource. The client sends `If-Match` with its update, and the server answers **412 Precondition Failed** if the version changed. That is how APIs stop two users from silently overwriting each other's edits.

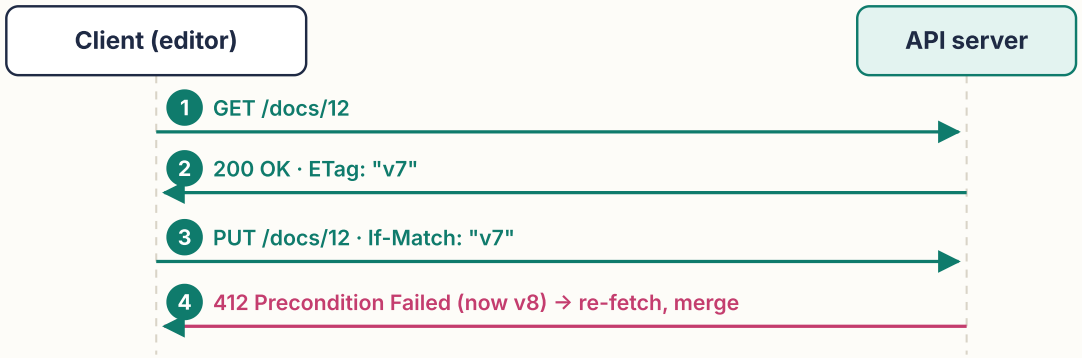


Figure 4. Someone else saved version 8 in between, so the PUT based on version 7 is refused instead of silently overwriting their changes.

COMMON MISTAKES

- Reading a value, changing it in code and writing it back, with no lock, version or atomic SQL.
- Using optimistic locking on one hot row: endless retries under load.
- Not checking the affected-row count, so a failed version check looks like success.
- Holding a `FOR UPDATE` lock while calling a payment provider.

6 Key terms

Lost update: a concurrent write silently overwrites another

Version column: a counter bumped on every update

Contention: how often writers hit the same row

Pessimistic locking: lock the row before reading (FOR UPDATE)

Compare-and-set: write only if the value is unchanged

ETag / If-Match: HTTP version check for an update

Optimistic locking: no lock; check a version on write

Atomic conditional update: SET n = n − 1 WHERE n > 0

412: Precondition Failed: version changed

7 Interview questions

Q1. When would you use optimistic locking? BASIC

When conflicts are rare, like profile or document edits. You hold no locks, check a version on write, and retry if it changed.

Q2. How do you avoid overselling stock? BASIC

Make the decrement atomic and conditional: UPDATE ... SET stock = stock − 1 WHERE id = ? AND stock > 0, and check the row count. Or lock the row with SELECT FOR UPDATE, or use a version check. For extreme hot spots, use Redis counters or a queue.

Q3. What does "0 rows updated" mean with a version check? BASIC

Someone changed the row after you read it, so your write was refused. Re-read the row, re-apply the change, and retry (usually with a small limit, e.g. 3 attempts).

Q4. Two support agents edit the same ticket in a web UI. How do you avoid lost edits? SCENARIO

Return an ETag (the version) with GET. Require If-Match on PUT/PATCH. If the version changed, return 412 Precondition Failed; the UI re-fetches and shows the other agent's changes before saving.

Q5. A flash sale's one product row gets 5,000 buy attempts per second. Which approach? SCENARIO

Not optimistic (retry storm). An atomic conditional UPDATE is the simplest correct choice. If the row itself becomes the bottleneck, keep the counter in Redis (atomic DECR), split stock across e.g. 10 bucket rows, or queue purchases.

Q6. Your bank transfer must check two balances before writing. Optimistic or pessimistic? SCENARIO

Pessimistic is simpler here: SELECT both accounts ORDER BY id FOR UPDATE, check, update, commit. Locking in ID order avoids deadlocks, and the transaction is short.

Q7. Is optimistic locking always faster because it takes no locks? TRICKY

No. Under high contention most attempts fail and retry, wasting work and adding latency. It wins only when conflicts are rare.

Q8. Does the atomic UPDATE take any lock? TRICKY

Yes, a row lock for the length of that one statement (and its transaction). Concurrent decrements of the same row still run one after another, but each holds the lock only briefly, with no application round trip inside.

Next: SD-E-013 · NoSQL Explained backendroadmap.com